

EDEN SOURCE GUIDE

Project Eden: Codebase Directory & File Guide

Detailed Folder and Component Explanations for Developers &
Academic Review



Course Module: CSE-401: Practical Software Architecture Audit

Version: v1.2.0

Date: July 2026

Classification: Academic Study Reference Guide

PROJECT EDEN: DIRECTORY & CODEBASE GUIDE

Comprehensive File-by-File Technical Guide for Exam Preparation

1. BACKEND STRUCTURE (/backend)

The backend of Project Eden is implemented as a modular Django application combined with Celery for background task processing. Below is the file-by-file breakdown of the backend:

1.1 Project Configuration & Celery Core (/backend/core)

- **`__init__.py`** : Tells Python to treat this directory as a package. It also imports the Celery app on project startup to ensure shared Celery tasks load correctly.
- **`settings.py`** : The central settings file for Django. Configures database setups (SQLite default), registers installed apps, configures REST framework, Celery brokers (Redis URL), cross-origin headers (CORS), and media file directories.
- **`urls.py`** : Root URL routing configuration. Directs API requests to the `/api/` paths and serves media assets in development mode.
- **`celery.py`** : Creates and configures the Celery instance. Discovers background tasks automatically across all registered Django apps.
- **`wsgi.py` / `asgi.py`** : Interfaces for production deployment (Web Server Gateway Interface and Asynchronous Server Gateway Interface).

1.2 Core Data Models (/backend/core_app)

- **`models.py`** : Defines the database schemas using Django ORM:
 - **`AnalysisJob`** : Tracks the state of the analysis pipeline (Pending, Downloading, Processing, Analyzing, Completed, Failed).
 - **`AnalysisReport`** : Stores the final aggregated structured JSON report data returned by the orchestrator.
 - **`MediaAsset`** : Tracks files created on disk during ingestion and processing (original video, audio wav track, frame screenshots, thumbnails).

- `ClaimRecord` : Stores individual claims, their verdict classifications (Likely True, Plausible, Unverified, High Risk, etc.), confidence score, reasoning text, and reference sources.
- `admin.py` : Registers models with the Django Admin site for easy monitoring.
- `apps.py` : Configurations for the core app initialization.
- `migrations/` : Automatically generated SQL schemas to keep the database in sync with the models in `models.py` .

1.3 REST API Endpoints (`/backend/api`)

- `views.py` : Contains the view controllers that process REST API requests:
 - `JobViewSet` : Handles creation (`POST /api/jobs/`) of URL analyses and file uploads, retrieves status updates (`GET /api/jobs/{id}/status/`), and fetches completed analysis reports.
- `serializers.py` : Translates Django model instances into JSON representations and validates incoming API request payloads.
- `urls.py` : Exposes REST endpoints to the router. Maps routes to the views in `views.py` .
- `management/commands/` : Custom administrative command scripts (e.g. commands to run specific tests or wipe databases).

1.4 Ingestion Service (`/backend/ingestion`)

- `services.py` : Houses the `InstagramIngestionService` :
 - Uses cookie-based authentication via Netscape formatted cookies to access Instagram.
 - Executes `yt-dlp` to download reels or video content.
 - Falls back to `instaloader` to download static image-based posts.
 - Falls back to `Playwright` headless Chromium browser to scrape and extract media URLs directly if `yt-dlp`/`instaloader` are blocked.
- `tasks.py` : Celery tasks wrapping the Ingestion Service to run in the background. Tracks status transitions in the database.
- `cache.py` : Implements `MediaCache` , which hashes URLs to check if the file has been downloaded previously, avoiding duplicate downloads and API queries.

1.5 Media Processing Layer (/backend/processing)

- **services.py** : Technical logic for media isolation and transcription:
 - Uses `FFmpeg` to extract audio streams from videos and convert them into 16kHz mono WAV format.
 - Runs OpenCV to extract video frames at 1 frame per second.
 - Applies CLAHE contrast enhancement on extracted frames.
 - Initializes `EasyOCR` readers to run character recognition on frames, filtering low-confidence tokens and deduplicating adjacent frames.
- **tasks.py** : Celery tasks that coordinate audio extraction, Whisper transcription, and OCR processing.

1.6 AI Analysis & Fact Verification Layer (/backend/analysis)

- **services.py** : Houses the AI Orchestration layer:
 - Defines Pydantic models `ClaimModel` and `ReportModel` to force external LLMs to return strict JSON data.
 - `GeminiProvider` : Connects to `google-genai` to run `gemini-2.5-flash`. Retrieves queries across flash-lite, flash-1.5, and pro models on rate limit triggers.
 - `HuggingFaceProvider` : Handles backups via hosted HuggingFace Inference endpoints (Llama 3, Mistral 7B).
 - `DegradedProvider` : Offline heuristic fallback model. Evaluates localized text matches, searches for sensationalist markers, and scores risk without an internet connection.
 - `AnalysisOrchestrator` : Sequences the providers from Gemini → Llama → Mistral → Offline Heuristic.
- **tasks.py** : Celery task that calls the orchestrator, and executes real-time web searches using BeautifulSoup scrapers on DuckDuckGo and Yahoo to verify claims.

1.7 Global Config & Assets

- **config/cookies.txt** : Active Instagram session cookies in Netscape format. Necessary to prevent Instagram login blocks.
- **manage.py** : Django CLI entrypoint for running the server, migrations, and shell.
- **requirements.txt** : Lists all python packages required for the project (Django, Celery, Playwright, EasyOCR, Whisper, etc.).
- **.env** : Stores environment variables (Gemini API keys, HuggingFace tokens, etc.).

2. FRONTEND STRUCTURE (/frontend)

The frontend is built as a single-page React application using Vite, configured with a dark-themed OSINT interface.

2.1 Src Core Layout & Routing (/frontend/src)

- **main.jsx** : The entrypoint that renders the React tree into the root DOM element.
- **App.jsx** : The core component that manages state and routing:
 - Uses `react-router-dom` to map routes (`/` and `/operation/:id`).
 - Injects a global Ctrl+K keystroke listener to toggle the Command Palette.
 - Maintains lists of historical runs using local hooks.
- **index.css** : Global layout styling, reset directives, and typography bindings.
- **tokens.css** : Central design system token definitions:
 - Colors: Base slate dark background (`#060608`), card backdrops (`#16161E`), translucent panels.
 - Status colors: Neon Green (`VERIFIED_LIKELY_TRUE`), Cyan/Blue (`PLAUSIBLE`), Amber/Orange (`UNVERIFIED`), Cyber Crimson/Rose (`HIGH_RISK` / `LIKELY_FALSE`).
 - Fonts: Monospace and system sans-serif mappings.

2.2 Modular UI Components (/frontend/src/components)

- **CommandBar.jsx** : The main interface for job submission:
 - Features a dual-tab layout: URL Submission (validating Instagram links) and File Upload (drag-and-drop file inputs).
 - Displays analysis mode selection buttons (Full, Text, Audio).
- **JobStatus.jsx** : Shows live status notifications for active runs:
 - Polls the status API endpoint in intervals.
 - Shows human-readable progress updates.
- **PipelineStepper.jsx** : A linear tracker showing Celery stages (Pending → Downloading → Processing → Analyzing → Done) with glowing status icons.
- **ReportDashboard.jsx** : The central coordinator for the Bento dashboard:
 - Arranges data into bento-grid layouts.

- Distributes analysis findings across visual widgets.
- **ClaimCard.jsx** : Cards showing fact-checked assertions:
 - Displays classification labels with corresponding neon background accents.
 - Renders confidence scores in a percentage bar.
- **CredibilityTimeline.jsx** : An interactive UI widget:
 - Synchronizes video timestamps with OCR transcriptions and audio overlays.
 - Allows users to click any timeline stamp to highlight corresponding visual frames.
- **DetailDrawer.jsx** : A side-panel that slides out when clicking a claim:
 - Displays LLM reasoning comments.
 - Lists search provenance and verification source links retrieved from web searches.
- **CommandPalette.jsx** : A modal search panel (⌘K):
 - Enables search inputs to query previous runs.
 - Provides quick commands to clear database records or purge history.
- **layout/AppShell.jsx** : Implements responsive page shells, with left navigation rails for desktop screens and tabs for mobile interfaces.
- **services/api.js** : Reusable Fetch API calls to Django REST endpoints (create job, upload media, retrieve job details, status logs).